

# Software e Engenharia de Software

CST em Análise e Desenvolvimento de Sistemas

Professor: Arliones Hoeller Jr.

[arliones.hoeller@ifsc.edu.br](mailto:arliones.hoeller@ifsc.edu.br)

# Licenciamento



Slides licenciados sob [Creative Commons](https://creativecommons.org/licenses/by/4.0/) “Atribuição 4.0 Internacional”

# Definição de Software

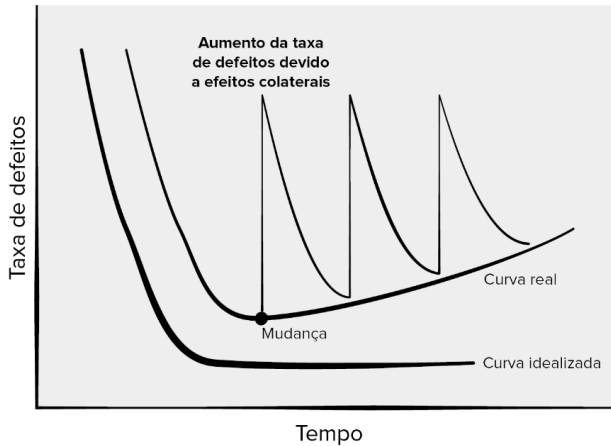
Software é:

- Instruções (programas de computador) que, quando executadas, fornecem recursos, funções e desempenho desejados;
- Estruturas de dados que permitem aos programas manipular adequadamente as informações;
- Documentação que descreve a operação e o uso dos programas.

# O que é Software?

- O software é desenvolvido ou projetado, não é fabricado no sentido clássico;
- O software não "se desgasta", mas pode deteriorar;
- Embora a indústria esteja migrando para a construção baseada em componentes, a maior parte do software continua sendo personalizado.

# Desgaste vs Deterioração



**Figura 1.2**  
Curva de defeitos para *software*.

Fonte: (PRESSMAN; MAXIM, 2021)

# Domínios de Aplicação do Software

- Software de sistema;
- Software aplicativo;
- Software de engenharia/científico;
- Software embarcado;
- Software de linha de produtos;
- Aplicações web/móveis;
- Software de IA (robótica, redes neurais, jogos).

# Software Legado

Por que o software precisa mudar?

- O software precisa ser adaptado para atender às necessidades de novos ambientes de computação ou tecnologia;
- O software deve ser aprimorado para implementar novos requisitos de negócios;
- O software deve ser estendido para torná-lo interoperável com outros sistemas ou bancos de dados mais modernos;
- O software deve ser reestruturado para torná-lo viável em um ambiente de rede.

# Definindo a Disciplina

Definição do IEEE: Engenharia de Software

- 1 Aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software, ou seja, aplicação de engenharia ao software;
- 2 O estudo de abordagens mencionadas no item 1.



# Camadas da Engenharia de Software



**Figura 1.3**

Camadas da engenharia de *software*.

Fonte: (PRESSMAN; MAXIM, 2021)

# Camadas da Engenharia de Software

- Engenharia de software é uma tecnologia em camadas.
- Foco na qualidade é a base de todas as camadas.
- Processos fornecem a estrutura para desenvolvimento.
- A melhoria contínua de processos leva ao aumento da eficácia.
- Métodos definem as tarefas técnicas e princípios.
- Ferramentas automatizam e integram processo e métodos.

# Atividades da Metodologia do Processo I

Uma metodologia (*framework*) de processo estabelece o alicerce para um processo de engenharia de software completo por meio da identificação de um pequeno número de atividades metodológicas aplicáveis a todos os projetos de software, independentemente de tamanho ou complexidade. (PRESSMAN; MAXIM, 2021, p. 10)

# Atividades da Metodologia do Processo II

## Cinco atividades de uma metodologia genérica:

- 1 **Comunicação:** entre equipe e com partes interessadas - é fundamental para entender os objetivos e coletar requisitos.
- 2 **Planejamento:** define tarefas técnicas, riscos prováveis, recursos necessários, artefatos a serem produzidos e cronograma.
- 3 **Modelagem:** criação de modelos que representam as necessidades do software e o projeto que vai atender essas necessidades.
- 4 **Construção:** produção de software - geração de código (manual ou automatizada) e testes.
- 5 **Entrega:** entrega do software ao cliente - instalação, suporte e manutenção.

# Atividades de Apoio I

A metodologia de processo engloba um conjunto de atividades de apoio (*umbrella activities*) aplicáveis a todo o processo de software. (PRESSMAN; MAXIM, 2021, p. 10)

# Atividades de Apoio II

## Atividades de apoio típicas:

- Controle e acompanhamento do projeto
- Administração de riscos
- Garantia da qualidade de software
- Revisões técnicas
- Medição
- Gerenciamento da configuração de software
- Gerenciamento da capacidade de reutilização
- Preparo e produção de artefatos de software

# Adaptação do Processo I

O processo de engenharia de software não é rígido nem deve ser seguido à risca. Ele deve ser ágil e adaptável ao problema, ao projeto, à equipe e à cultura organizacional. (PRESSMAN; MAXIM, 2021, p. 11)

# Adaptação do Processo II

## Potenciais diferenças entre processos:

- Fluxo geral de atividades, ações e tarefas e suas interdependências.
- Até que ponto as ações e tarefas são definidas dentro de cada atividade.
- Até que ponto artefatos de software são identificados e exigidos.
- Modo de aplicar as atividades de garantia da qualidade.
- Modo de aplicar as atividades de acompanhamento e controle do projeto.
- Grau geral de detalhamento e rigor da descrição do processo.
- Grau de envolvimento por parte do cliente ou de outros envolvidos.
- Nível de autonomia dada à equipe de software.
- Grau de prescrição da organização da equipe.



# Atividades Essenciais da Engenharia de Software

A solução de problemas é a essência da prática da engenharia de software. Segundo Polya (1978), a essência da solução de problemas consiste em:

- 1 Entender o problema (comunicação e análise);
- 2 Planejar uma solução (modelagem e projeto de software);
- 3 Executar o plano (geração de código);
- 4 Examinar o resultado para ter precisão (testes e garantia de qualidade).

# Entender o Problema

Algumas perguntas simples que ajudam a entender o problema:

- *Quem tem interesse na solução do problema?*  
Quem são os envolvidos (*stakeholders*)?
- *Quais são as incógnitas?*  
Quais dados, funções e recursos são necessários para resolver apropriadamente o problema?
- *O problema pode ser compartimentalizado?*  
É possível representá-lo em problemas menores que talvez sejam mais fáceis de ser compreendidos?
- *O problema pode ser representado graficamente?*  
É possível criar um modelo analítico?

# Planejar uma Solução

## Planejar antes de fazer.

Perguntas que podem auxiliar o planejamento:

- *Você já viu problemas semelhantes anteriormente?*  
Há padrões que reconhecíveis em uma possível solução? Há software que implemente os dados, as funções e as características necessárias?
- *Algum problema semelhante já foi resolvido?*  
Se sim, os elementos da solução são reutilizáveis?
- *É possível definir subproblemas?*  
Se sim, há soluções aparentes e imediatas para eles?
- *É possível representar uma solução de maneira que conduza a uma implementação efetiva?*  
É possível criar um modelo de projeto?

# Executar o Plano

O projeto é um mapa do caminho a ser seguido na construção do sistema. Podem surgir desvios, ou até serem encontrados melhores caminhos, mas o planejamento evita que se perca tempo e recursos.

- *A solução é adequada ao plano?*

O código-fonte pode ser atribuído ao modelo do projeto?

- *Todas as partes componentes da solução estão provavelmente corretas?*

O projeto e o código foram revisados? Melhor ainda, provas de correção foram aplicadas?

# Examinar o Resultado

Não se pode ter certeza de que a solução seja perfeita, mas pode-se assegurar de que ela tenha sido suficientemente testada para revelar o maior número de erros possível antes da entrega.

- *É possível testar cada parte da solução?*  
Foi implementada uma estratégia de testes razoável?
- *A solução produz resultados adequados aos dados, às funções e às características necessárias?*  
O software foi validado em relação a todas solicitações dos envolvidos?

# Regra de Ouro da Engenharia de Software

**Bom Senso!**

# Princípios Gerais de Hooker

Hooker (1996) propõe sete princípios gerais para processos de software:

- 1 **A razão de tudo existir:** agregar valor aos usuários.
- 2 **Mantenha simples:** o projeto deve ser o mais simples possível (*KISS – Keep It Simple, Stupid!*).
- 3 **Mantenha a visão:** uma visão clara é essencial.
- 4 **O que um produz produz, Outros consomem.**
- 5 **Esteja aberto ao futuro:** não projete algo que o trave no futuro.
- 6 **Planeje para reutilização:** isso reduz custos e aumenta o valor.
- 7 **Pense!:** colocar o pensamento antes da ação produz resultados melhores.

# Resumo

- Software é um conjunto de instruções, estruturas de dados e documentação que, quando executados, fornecem recursos, funções e desempenho desejados.
- A engenharia de software é a aplicação de uma abordagem sistemática, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software.
- Engenharia de software abrange processos, métodos e ferramentas para construir sistemas complexos com qualidade.
- A solução de problemas é a essência da prática da engenharia de software.
- O processo de engenharia de software não é rígido e deve ser adaptado ao problema, ao projeto, à equipe e à cultura organizacional.



# Questões para debater I

- 1 Forneça uma série de exemplos (positivos e negativos) que indiquem o impacto do software em nossa sociedade.
- 2 Dê suas próprias respostas para as cinco perguntas abaixo. Discuta-as com seus colegas.
  - Por que a conclusão de um software leva tanto tempo?
  - Por que os custos de desenvolvimento são tão altos?
  - Por que não conseguimos encontrar todos os erros antes de entregarmos o software aos clientes?
  - Por que gastamos tanto tempo e esforço realizando a manutenção de programas existentes?
  - Por que ainda temos dificuldades de medir o progresso de desenvolvimento e a manutenção de um software?

## Questões para debater II

- 3 Considere as sete categorias de software apresentadas no Slide 5. Você acha que a mesma abordagem em relação à engenharia de software pode ser aplicada a cada uma delas? Justifique sua resposta.
- 4 Descreva uma metodologia de processo com suas próprias palavras. Ao afirmarmos que atividades de modelagem se aplicam a todos os projetos, isso significa que as mesmas tarefas são aplicadas a todos os projetos, independentemente de seu tamanho e complexidade? Explique.
- 5 As atividades de apoio ocorrem ao longo do processo de software. Você acredita que elas são aplicadas de forma homogênea ao longo do processo ou algumas delas são concentradas em uma ou mais atividades da metodologia?

# Referências

-  HOOKER, David. **Seven Principles of Software Development**. 1996.  
Disponível em: <<https://wiki.c2.com/?SevenPrinciplesOfSoftwareDevelopment>>.
-  POLYA, George. **A Arte de Resolver Problemas**. 1. ed.: Interciência, 1978.
-  PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. Porto Alegre: Bookman, 2021.  
<https://app.minhabiblioteca.com.br/books/9786558040118/>.